Beginner Handout: Understanding CNN Convolution and Pooling

CSE465.2: Pattern Recognition

Dr. Sumaiya Tabassum Nimi
Assistant Professor, Dept. of ECE — North South University

Introduction

In this handout, we will learn how Convolutional Neural Networks (CNNs) perform:

- Convolution
- Pooling

We will go step-by-step with a full numerical example.

Basic Terminology

- **Input Matrix**: The image (or feature map)
- **Kernel (Filter)**: Small matrix used to scan the input
- **Stride**: Number of steps the filter moves
- **Padding**: Adding zeros around input (not used in this example)

Example Setup

Input Matrix (5x5)

$$I = \begin{bmatrix} 1 & 2 & 0 & 3 & 1 \\ 4 & 1 & 2 & 1 & 0 \\ 0 & 1 & 3 & 1 & 2 \\ 2 & 0 & 1 & 3 & 1 \\ 1 & 2 & 1 & 0 & 2 \end{bmatrix}$$

Kernel (3x3)

$$K = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Parameters

- Stride = 1
- Padding = 0 (valid convolution)

Output Size Calculation

$$\text{Output Size} = \frac{N - F}{S} + 1$$

Where:

- $N = 5$ (input size)
- $F = 3$ (filter size)
- $S = 1$ (stride)

$$\text{Output} = \frac{5 - 3}{1} + 1 = 3$$

So output will be 3×3.

Step-by-Step Convolution

Step 1 (Top-left)

$$\begin{bmatrix} 1 & 2 & 0 \\ 4 & 1 & 2 \\ 0 & 1 & 3 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Element-wise multiplication:

$$(1 \times 1) + (2 \times 0) + (0 \times 1) + (4 \times 0) + (1 \times 1) + (2 \times 0) + (0 \times 1) + (1 \times 0) + (3 \times 1)$$

$$= 1 + 0 + 0 + 0 + 1 + 0 + 0 + 0 + 3 = 5$$

Step 2 (Shift Right)

$$\begin{bmatrix} 2 & 0 & 3 \\ 1 & 2 & 1 \\ 1 & 3 & 1 \end{bmatrix}$$

$$= (2 \times 1) + (0 \times 0) + (3 \times 1) + (1 \times 0) + (2 \times 1) + (1 \times 0) + (1 \times 1) + (3 \times 0) + (1 \times 1)$$

$$= 2 + 0 + 3 + 0 + 2 + 0 + 1 + 0 + 1 = 9$$

Step 3 (Shift Right Again)

$$\begin{bmatrix} 0 & 3 & 1 \\ 2 & 1 & 0 \\ 3 & 1 & 2 \end{bmatrix}$$

$$= (0 \times 1) + (3 \times 0) + (1 \times 1) + (2 \times 0) + (1 \times 1) + (0 \times 0) + (3 \times 1) + (1 \times 0) + (2 \times 1)$$

$$= 0 + 0 + 1 + 0 + 1 + 0 + 3 + 0 + 2 = 7$$

Continue Similarly...

After computing all positions:

$$\text{Feature Map} = \begin{bmatrix} 5 & 9 & 7 \\ 6 & 8 & 6 \\ 4 & 7 & 8 \end{bmatrix}$$

Pooling Layer (Max Pooling)

Parameters

- Pool Size $= 2 \times 2$
- Stride $= 2$

Step-by-Step

Take top-left 2×2:

$$\begin{bmatrix} 5 & 9 \\ 6 & 8 \end{bmatrix} \Rightarrow \max = 9$$

Next:

$$\begin{bmatrix} 7 & - \\ 6 & - \end{bmatrix} \Rightarrow \max = 7$$

(Only valid region considered)

Bottom:

$$\begin{bmatrix} 4 & 7 \\ - & - \end{bmatrix} \Rightarrow \max = 7$$

Final Output

$$\text{Pooled Output} = \begin{bmatrix} 9 & 7 \\ 7 & 8 \end{bmatrix}$$

Key Takeaways

- Convolution extracts features using filters
- Stride controls movement
- Pooling reduces size and keeps important information
- CNN builds understanding layer by layer

Convolution with Padding

What is Padding?

Padding means adding zeros around the border of the input matrix.

- Helps preserve spatial size
- Allows edge pixels to be used more effectively

Example Setup

We use the same input and kernel:

$$I = \begin{bmatrix} 1 & 2 & 0 & 3 & 1 \\ 4 & 1 & 2 & 1 & 0 \\ 0 & 1 & 3 & 1 & 2 \\ 2 & 0 & 1 & 3 & 1 \\ 1 & 2 & 1 & 0 & 2 \end{bmatrix}$$

Kernel:

$$K = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Parameters

- Stride = 1
- Padding = 1

Step 1: Apply Padding

We add one layer of zeros around the input:

$$I_{padded} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 & 3 & 1 & 0 \\ 0 & 4 & 1 & 2 & 1 & 0 & 0 \\ 0 & 0 & 1 & 3 & 1 & 2 & 0 \\ 0 & 2 & 0 & 1 & 3 & 1 & 0 \\ 0 & 1 & 2 & 1 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Step 2: Output Size

$$\text{Output} = \frac{N + 2P - F}{S} + 1$$

$$= \frac{5 + 2(1) - 3}{1} + 1 = 5$$

So output is 5×5.

Step 3: First Convolution (Top-left)

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 4 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

$$= (0 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (2 \times 0) + (0 \times 1) + (4 \times 0) + (1 \times 1)$$

$$= 0 + 0 + 0 + 0 + 1 + 0 + 0 + 0 + 1 = 2$$

Step 4: Move Right

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 2 & 0 \\ 4 & 1 & 2 \end{bmatrix}$$

$$= (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (2 \times 1) + (0 \times 0) + (4 \times 1) + (1 \times 0) + (2 \times 1)$$

$$= 0 + 0 + 0 + 0 + 2 + 0 + 4 + 0 + 2 = 8$$

Continue for All Positions

After computing all positions, the final output becomes:

$$\text{Feature Map with Padding} = \begin{bmatrix} 2 & 8 & 5 & 7 & 1 \\ 6 & 5 & 9 & 7 & 3 \\ 4 & 6 & 8 & 6 & 4 \\ 3 & 4 & 7 & 8 & 2 \\ 1 & 3 & 4 & 2 & 2 \end{bmatrix}$$

Key Insight

- Without padding: Output shrinks ($5 \to 3$)
- With padding: Output size preserved ($5 \to 5$)
- Edge information is retained

Using Multiple Filters in Convolution

Why Use Multiple Filters?

In practice, a CNN usually does not use only one filter. Instead, it uses **many filters**.

Each filter learns to detect a different kind of pattern, such as:

- edges
- corners
- textures
- simple shapes

Important idea:

- One filter $\to$ one feature map
- Multiple filters $\to$ multiple feature maps
- Stacking all feature maps together $\to$ output volume

Recall the Input

Suppose the input is the same 5×5 matrix:

$$I = \begin{bmatrix} 1 & 2 & 0 & 3 & 1 \\ 4 & 1 & 2 & 1 & 0 \\ 0 & 1 & 3 & 1 & 2 \\ 2 & 0 & 1 & 3 & 1 \\ 1 & 2 & 1 & 0 & 2 \end{bmatrix}$$

Assume:

- stride $= 1$
- padding $= 0$
- filter size $= 3 \times 3$

So each filter will produce a 3×3 feature map.

Filter 1

Let the first filter be:

$$K_1 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

From our previous calculation, its output feature map is:

$$F_1 = \begin{bmatrix} 5 & 9 & 7 \\ 6 & 8 & 6 \\ 4 & 7 & 8 \end{bmatrix}$$

Filter 2

Now suppose we use a second filter:

$$K_2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

This filter focuses on a different pattern.

Step-by-Step Calculation for Filter 2

Top-left position

Take the first 3×3 window from the input:

$$\begin{bmatrix} 1 & 2 & 0 \\ 4 & 1 & 2 \\ 0 & 1 & 3 \end{bmatrix}$$

Now apply element-wise multiplication with K_2:

$$\begin{bmatrix} 1 & 2 & 0 \\ 4 & 1 & 2 \\ 0 & 1 & 3 \end{bmatrix} \odot \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$= (1 \times 0) + (2 \times 1) + (0 \times 0) + (4 \times 1) + (1 \times 0) + (2 \times 1) + (0 \times 0) + (1 \times 1) + (3 \times 0)$$

$$= 0 + 2 + 0 + 4 + 0 + 2 + 0 + 1 + 0 = 9$$

Move one step right

Next window:

$$\begin{bmatrix} 2 & 0 & 3 \\ 1 & 2 & 1 \\ 1 & 3 & 1 \end{bmatrix}$$

$$= (2 \times 0) + (0 \times 1) + (3 \times 0) + (1 \times 1) + (2 \times 0) + (1 \times 1) + (1 \times 0) + (3 \times 1) + (1 \times 0)$$

$$= 0 + 0 + 0 + 1 + 0 + 1 + 0 + 3 + 0 = 5$$

Move one step right again

Next window:

$$\begin{bmatrix} 0 & 3 & 1 \\ 2 & 1 & 0 \\ 3 & 1 & 2 \end{bmatrix}$$

$$= (0 \times 0) + (3 \times 1) + (1 \times 0) + (2 \times 1) + (1 \times 0) + (0 \times 1) + (3 \times 0) + (1 \times 1) + (2 \times 0)$$

$$= 0 + 3 + 0 + 2 + 0 + 0 + 0 + 1 + 0 = 6$$

Second row, first column

Window:

$$\begin{bmatrix} 4 & 1 & 2 \\ 0 & 1 & 3 \\ 2 & 0 & 1 \end{bmatrix}$$

$$= (4 \times 0) + (1 \times 1) + (2 \times 0) + (0 \times 1) + (1 \times 0) + (3 \times 1) + (2 \times 0) + (0 \times 1) + (1 \times 0)$$

$$= 0 + 1 + 0 + 0 + 0 + 3 + 0 + 0 + 0 = 4$$

Second row, second column

Window:

$$\begin{bmatrix} 1 & 2 & 1 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix}$$

$$= (1 \times 0) + (2 \times 1) + (1 \times 0) + (1 \times 1) + (3 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (3 \times 0)$$

$$= 0 + 2 + 0 + 1 + 0 + 1 + 0 + 1 + 0 = 5$$

Second row, third column

Window:

$$\begin{bmatrix} 2 & 1 & 0 \\ 3 & 1 & 2 \\ 1 & 3 & 1 \end{bmatrix}$$

$$= (2 \times 0) + (1 \times 1) + (0 \times 0) + (3 \times 1) + (1 \times 0) + (2 \times 1) + (1 \times 0) + (3 \times 1) + (1 \times 0)$$

$$= 0 + 1 + 0 + 3 + 0 + 2 + 0 + 3 + 0 = 9$$

Third row, first column

Window:

$$\begin{bmatrix} 0 & 1 & 3 \\ 2 & 0 & 1 \\ 1 & 2 & 1 \end{bmatrix}$$

$$= (0 \times 0) + (1 \times 1) + (3 \times 0) + (2 \times 1) + (0 \times 0) + (1 \times 1) + (1 \times 0) + (2 \times 1) + (1 \times 0)$$

$$= 0 + 1 + 0 + 2 + 0 + 1 + 0 + 2 + 0 = 6$$

Third row, second column

Window:

$$\begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}$$

$$= (1 \times 0) + (3 \times 1) + (1 \times 0) + (0 \times 1) + (1 \times 0) + (3 \times 1) + (2 \times 0) + (1 \times 1) + (0 \times 0)$$

$$= 0 + 3 + 0 + 0 + 0 + 3 + 0 + 1 + 0 = 7$$

Third row, third column

Window:

$$\begin{bmatrix} 3 & 1 & 2 \\ 1 & 3 & 1 \\ 1 & 0 & 2 \end{bmatrix}$$

$$= (3 \times 0) + (1 \times 1) + (2 \times 0) + (1 \times 1) + (3 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 1) + (2 \times 0)$$

$$= 0 + 1 + 0 + 1 + 0 + 1 + 0 + 0 + 0 = 3$$

So the output of the second filter is:

$$F_2 = \begin{bmatrix} 9 & 5 & 6 \\ 4 & 5 & 9 \\ 6 & 7 & 3 \end{bmatrix}$$

Stacking the Outputs

Now we have two feature maps:

$$F_1 = \begin{bmatrix} 5 & 9 & 7 \\ 6 & 8 & 6 \\ 4 & 7 & 8 \end{bmatrix}, \qquad F_2 = \begin{bmatrix} 9 & 5 & 6 \\ 4 & 5 & 9 \\ 6 & 7 & 3 \end{bmatrix}$$

These are stacked together to form the output volume.

$$\text{Output shape} = 3 \times 3 \times 2$$

This means:

- height $= 3$
- width $= 3$
- depth $= 2$ because we used 2 filters

Visual Interpretation

You can imagine it as:

$$\text{Output Volume} = \begin{bmatrix} F_1 \\ F_2 \end{bmatrix}$$

This does **not** mean writing one matrix under the other in 2D. It means they are stacked in the **depth direction**.

General Rule

If:

- input size is $N \times N$
- filter size is $F \times F$
- padding is P
- stride is S
- number of filters is M

then:

$$\text{output height} = \text{output width} = \frac{N - F + 2P}{S} + 1$$

and the full output shape is:

$$\left(\frac{N - F + 2P}{S} + 1\right) \times \left(\frac{N - F + 2P}{S} + 1\right) \times M$$

Example Summary

In our example:

- input $= 5 \times 5$
- filter size $= 3 \times 3$
- padding $= 0$
- stride $= 1$
- number of filters $= 2$

So:

$$\text{output height} = \text{output width} = \frac{5 - 3 + 0}{1} + 1 = 3$$

Since we used 2 filters:

$$\text{final output shape} = 3 \times 3 \times 2$$

Key Takeaways

- Each filter produces one feature map.
- Different filters detect different patterns.
- More filters mean greater depth in the output.
- The number of filters determines the number of channels in the output.

Number of Learnable Parameters in CNN

What are Learnable Parameters?

Learnable parameters are the values that the CNN learns during training.
In a convolution layer, these include:

- Filter (kernel) weights
- Bias terms

Key Idea

Each filter has:

- $F \times F$ weights (for single-channel input)
- $+1$ bias

If we use multiple filters, the total parameters increase.

Case 1: Single Filter, Single Channel Input

Suppose:

- Filter size $= 3 \times 3$
- Number of filters $= 1$
- Input channels $= 1$ (grayscale image)

Number of weights

$$3 \times 3 = 9$$

Add bias

$$9 + 1 = 10$$

$$\boxed{\text{Total parameters} = 10}$$

Case 2: Multiple Filters, Single Channel Input

Suppose:

- Filter size $= 3 \times 3$
- Number of filters $= 2$
- Input channels $= 1$

Each filter has:

$$9 + 1 = 10$$

So total:

$$2 \times 10 = 20$$

$$\boxed{\text{Total parameters} = 20}$$

Case 3: Multi-Channel Input (Important)

Now consider a color image (RGB).

- Input channels $= 3$
- Filter size $= 3 \times 3$

Each filter now spans all channels:

$$\text{Filter size} = 3 \times 3 \times 3$$

Weights per filter

$$3 \times 3 \times 3 = 27$$

Add bias

$$27 + 1 = 28$$

Case 4: Multiple Filters + Multi-Channel Input

Suppose:

- Input channels = 3
- Filter size = 3×3
- Number of filters = 4

Each filter has:

$$3 \times 3 \times 3 + 1 = 28$$

Total parameters:

$$4 \times 28 = 112$$

$$\boxed{\text{Total parameters} = 112}$$

General Formula

If:

- Filter size = $F \times F$
- Input channels = C
- Number of filters = M

Then:

$$\text{Parameters per filter} = F \times F \times C + 1$$

$$\text{Total parameters} = M \times (F \times F \times C + 1)$$

Important Observations

- Parameters **do not depend on input width/height**
- They depend only on:
  - filter size
  - number of filters
  - number of input channels
- This is why CNNs are **parameter efficient**

Comparison with Fully Connected Layer

Suppose:

- Input $= 5 \times 5 = 25$ values
- Fully connected layer with 10 neurons

$$25 \times 10 + 10 = 260$$

Compare with CNN:

Only 10 parameters (for one filter)

CNN uses far fewer parameters!

Key Takeaways

- Each filter has its own weights and bias
- More filters $\Rightarrow$ more parameters
- More channels $\Rightarrow$ more parameters per filter
- CNNs are efficient because weights are shared across space